

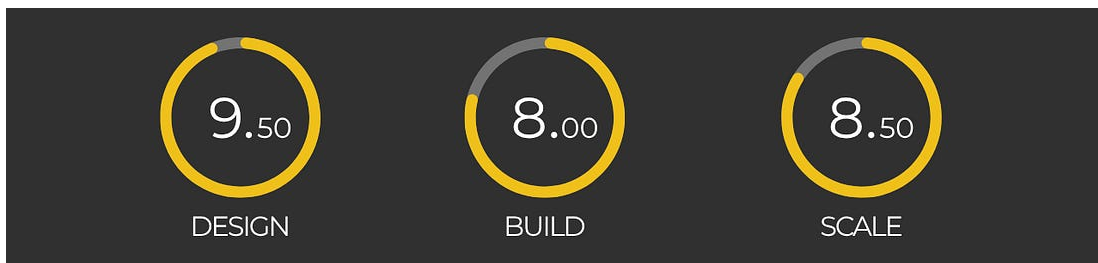
How Stock Exchange Processes 6 Million Events per Second with Microsecond Latency

#104: Stock Exchange System Design - Part 2



NEO KIM

NOV 29, 2025 PAID

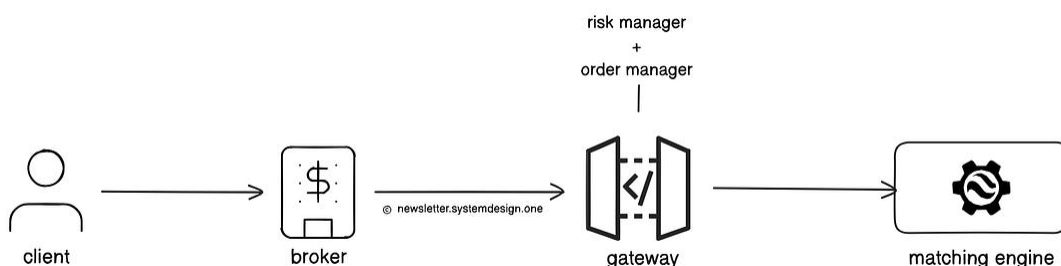


- *[Share this post](#) & I'll send you some rewards for the referrals.*
- *I created block diagrams for this newsletter using [Eraser](#).*

This month, I launched **Design, Build, Scale**... the newsletter series that will elevate your software engineering career.

This is Part 2 of how a stock exchange works.

Let's start with a **TL;DR of Part 1**!



Three key components of an exchange are:

- Broker - allows people to buy and sell stocks on the exchange.
- Gateway - entry point for brokers to send BUY or SELL orders to the exchange.
- Matching Engine - a component that matches BUY and SELL orders to create trades.

A stock market reduces latency by avoiding unnecessary services in the critical path.

BUT... threads have to pass events efficiently to achieve “ultra-low” latency at scale.

Examples:

- New order event,
- Risk check event,
- Cancel order event,
- Trade execution event,
- Market data update event,
- Replication or Journal event.

So the concurrency model matters!

Onward.

EverMemOS—The Next-Generation AI Memory System (Sponsor)



Most AI agents forget everything after a session—making them inconsistent, hard to debug, and impossible to scale.

Inspired by human brain memory mechanisms, **EverMemOS** provides an open-source memory OS that supports 1-on-1 conversation scenarios and complex multi-agent workflows. As reported, EverMemOS achieved 92.4% on LoCoMo and 82% on LongMemEval-S, both SOTA results of the two benchmarks.

Later this year, EverMind will launch the cloud service version, offering enterprise users advanced technical support, persistent storage, and scalable infrastructure.

If you're building agentic apps, EverMemOS gives you the memory layer you've been missing.

 [Explore EverMemOS](#)

Most system design goo-roos think using locks, or queues, or message brokers is enough.

They're WRONG...

Here's why:

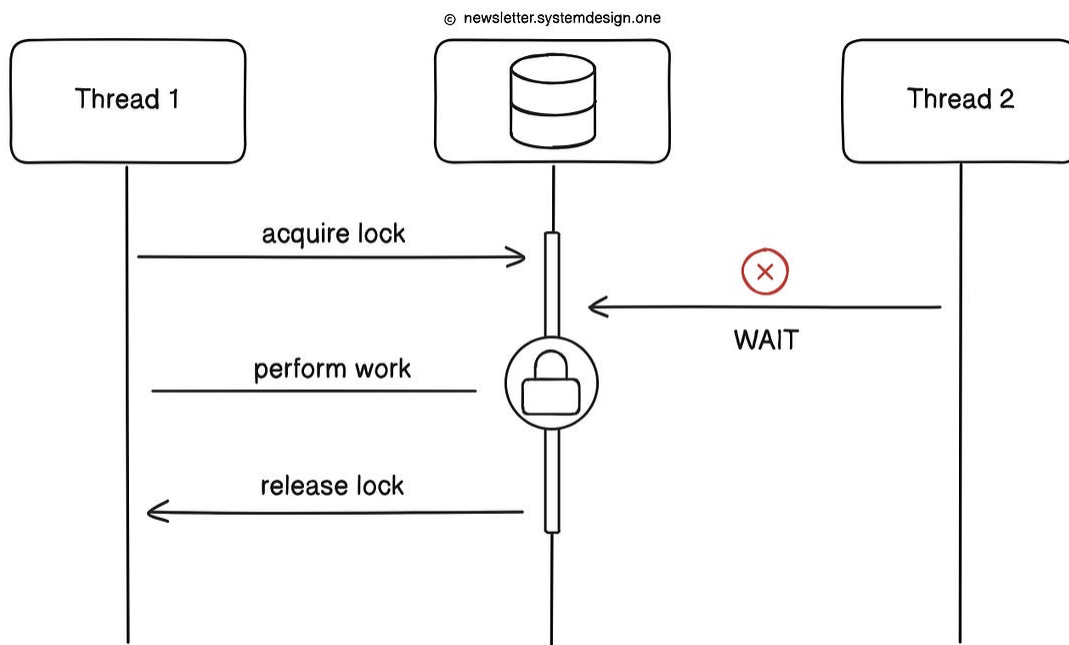
1 Locking

There are two key locking models:

- Pessimistic,
- Optimistic.

Pessimistic locking

A thread acquires a lock before touching shared data. So other threads must wait until the lock gets released. This means only one thread can update the shared data at a time.



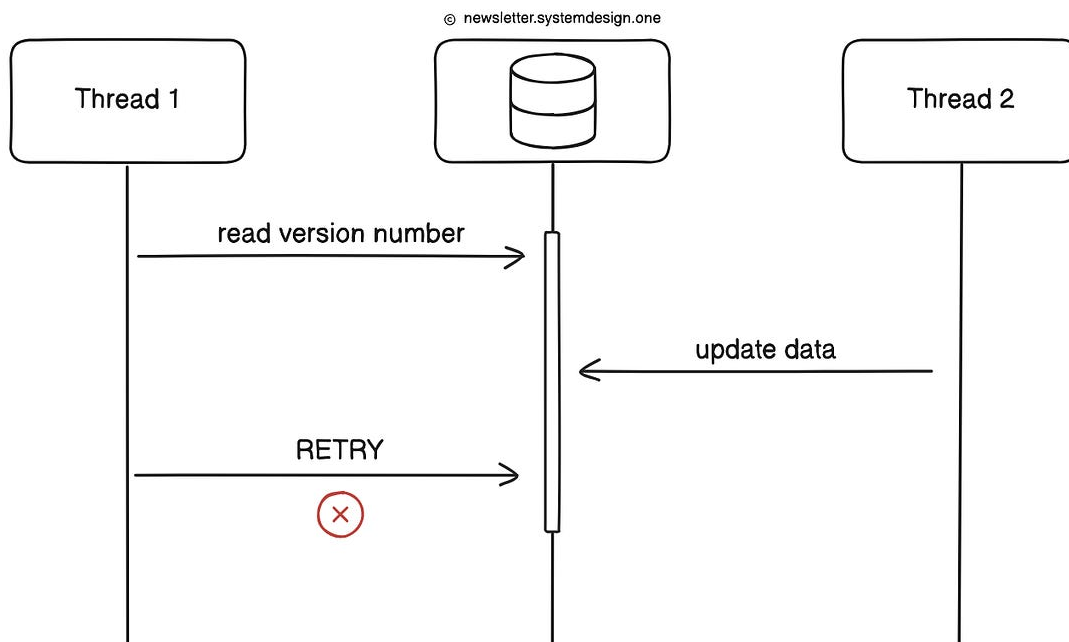
But on a stock exchange:

- Many threads try to read or update the same data at scale.
- And locks force them to wait in line, which causes latency spikes.
- More threads → more blocking → much SLOWER.
- Even a single lock can slow down a thread by 10x, and with many threads, performance could drop 100x.
- Besides, there's also a risk of deadlock¹ if two threads wait on each other.

So this approach wouldn't work for a stock exchange.

Optimistic locking

It doesn't use traditional locks; instead, it uses compare-and-swap (**CAS**) to detect conflicts.



Here's how:

1. A thread reads shared data + its version.
2. Then it writes new data only if the original version hasn't changed in the meantime.

If another thread updates it first, the write fails, and the thread has to RETRY.

This technique works well when conflicts are rare,,, but on a stock exchange:

- Many threads try to change hot data (sequence counters, and so on).
- Thus CAS failures become more frequent, and spin + retry loops waste CPU.

So performance weakens at scale.

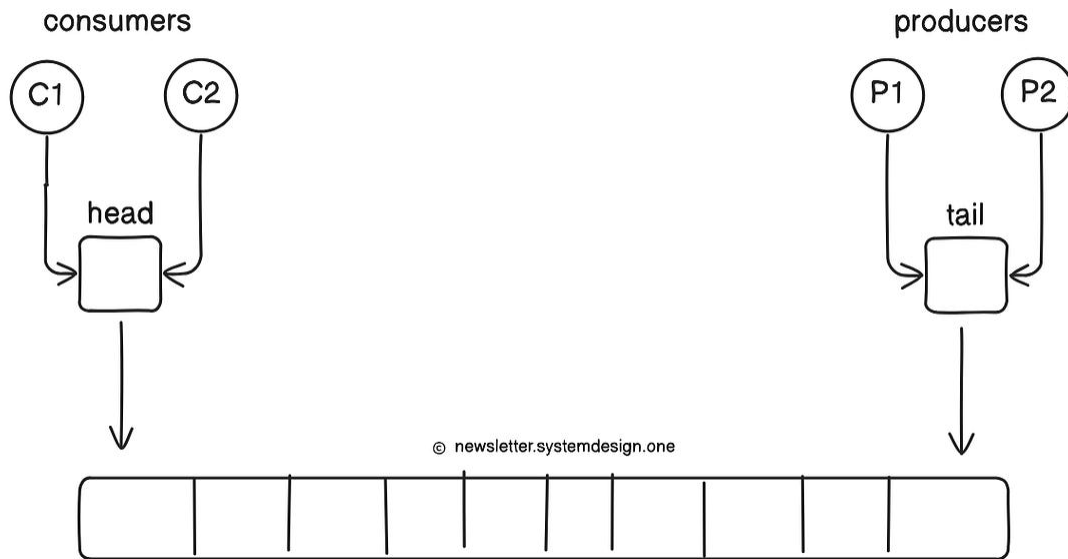
2 Traditional Queues (ArrayBlockingQueue)

A Queue has:

- Head (where consumers read from)
- Tail (where producers write to)

Yet when there are many threads at once:

- Consumers compete to update the head,
- Producers compete to update the tail.



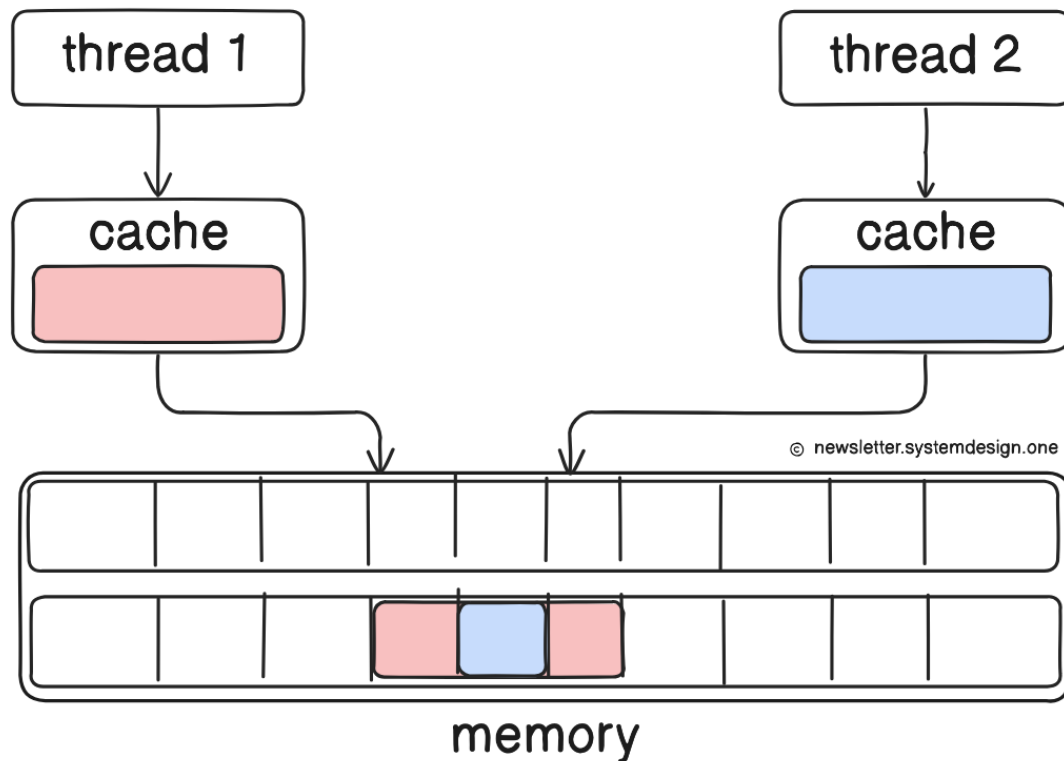
This creates contention and forces locking or other synchronization techniques... which slows everything down.

Plus, head, tail, and size variables are often on the same CPU cache line².

NOTE: A CPU reads and writes memory in chunks (usually 64 bytes) called **cache lines**, not single variables.

So there's a risk of **false sharing**³ when:

- Two threads update different variables
- And those variables live on the same cache line



Even though the threads aren't sharing data (i.e., no locks), the hardware thinks they're, causing:

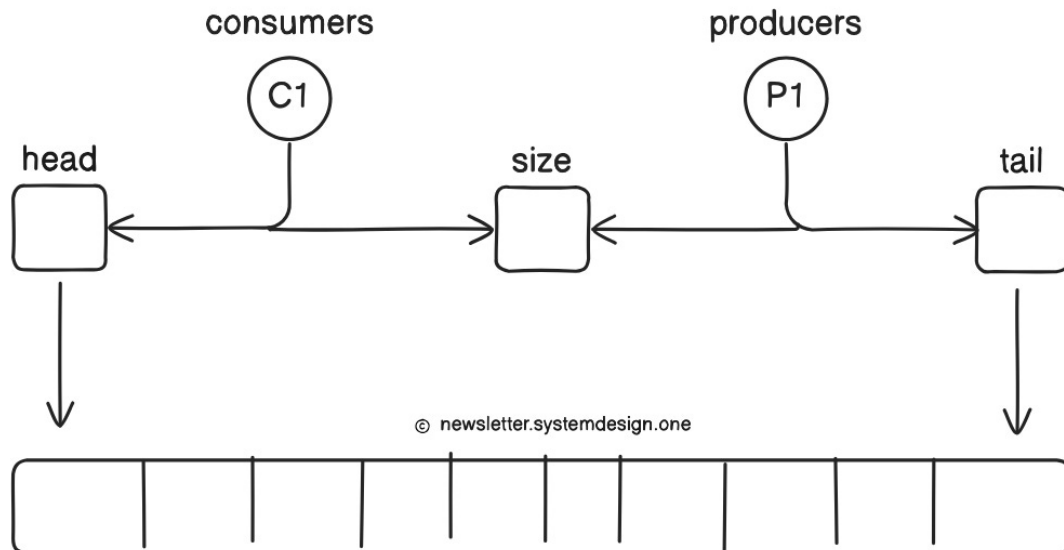
- massive slowdowns
- constant cache invalidations

This is a performance problem for traditional queues under high concurrency.

Also, a queue is often:

- Full (because producers are fast)
- Or empty (because consumers are fast)

So producers/consumers constantly fight over the same memory region. And this causes contention.



Besides, in some implementations, reads may require writes when consumed. This also causes contention.

3 Message Brokers

Message brokers like Kafka and RabbitMQ are built for general messaging... NOT for microsecond latency.

They introduce extra overhead because of:

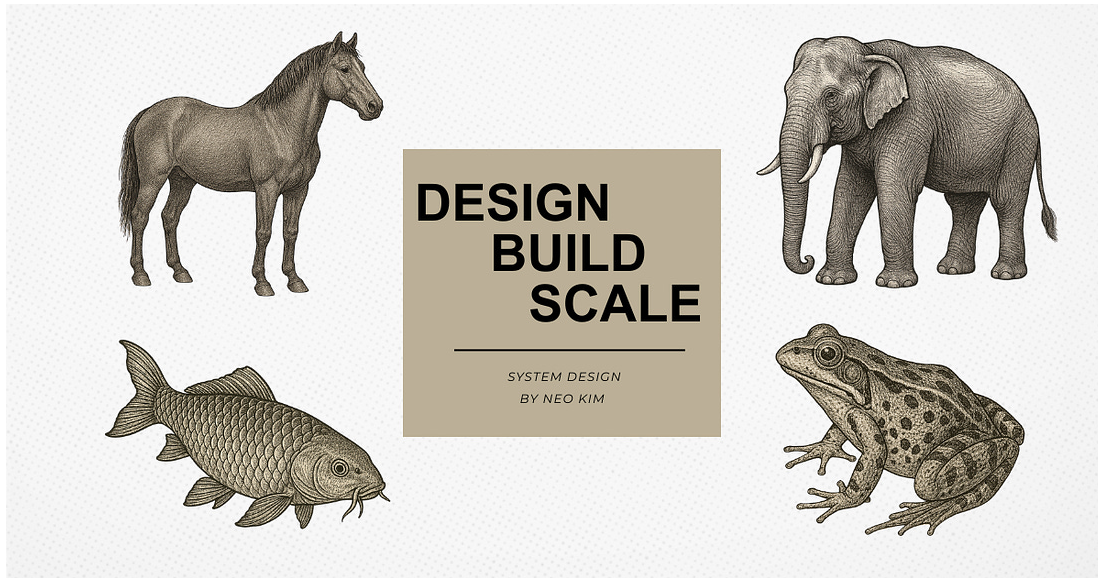
- serialization,
- buffering,
- network hops.

Plus, their queues use locks and background threads, which add jitter.

So they're too slow for a stock exchange!

Then how do you achieve "ultra-low" latency at SCALE?

Reminder: this is a teaser of the subscriber-only newsletter series, exclusive to my golden members.



When you upgrade, you'll get:

- **High-level architecture of real-world systems.**
- Deep dive into how popular real-world systems actually work.
- **How real-world systems handle scale, reliability, and performance.**

✓ Subscribed

Disruptor Pattern

A stock exchange needs a “gold standard” for concurrent communication between threads.

i.e., no locks, no blocking, no false sharing.

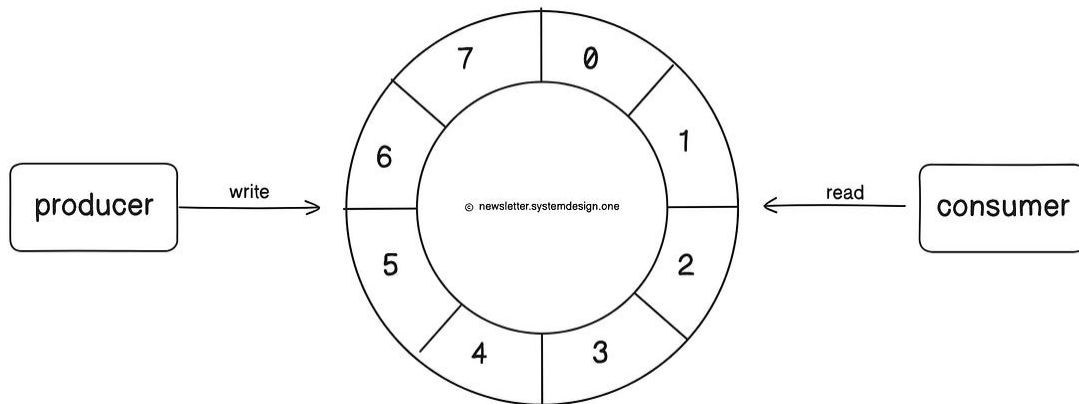
So they use the disruptor pattern:

"It's a concurrency model that doesn't use locks."

First things last... before we learn the disruptor, let's take a step back... and understand the **ring buffer**.

- It's an array arranged in a circle,
- And it provides threads a faster way⁴ to share data.

When you reach the end, it simply wraps around to the beginning (circular form).



Here's how you can find an array index on a ring buffer:

```
index = sequence_number % array_length
```

Plus, the array size is usually a power of two to speed up calculations⁵. Also, items stay in the buffer until they get overwritten (unlike a queue) when the sequence number wraps.

Now, let's learn **how the disruptor pattern works**:

- It uses the ring buffer: a fixed-size circular array, pre-allocated with event objects⁶.
- No shared tail/head pointers exist, like in a traditional queue.
- Instead, each producer and consumer maintains its own SEQUENCE COUNTER... think of it as the position marker.
- A producer writes an event into a slot⁷ that corresponds to its sequence number. Yet only one producer writes to a slot to avoid write contention.
- Consumers don't update slots. This avoids a fight between producers and consumers over slots.
- Instead, each consumer tracks its own (read) sequence number. And can run in "parallel"⁸ without blocking each other.
- They also check other consumers' sequences to ensure ordering when required.
- Besides, consumers process events in batches⁹, thus reducing latency and increasing throughput.

As only ONE producer writes to the disruptor, there is no:

- locks,
- contention,
- cache invalidation.

Thus, achieving mechanical sympathy¹⁰.

So what happens when there are many producers?

Then, those producers share a SINGLE sequence counter.

How?

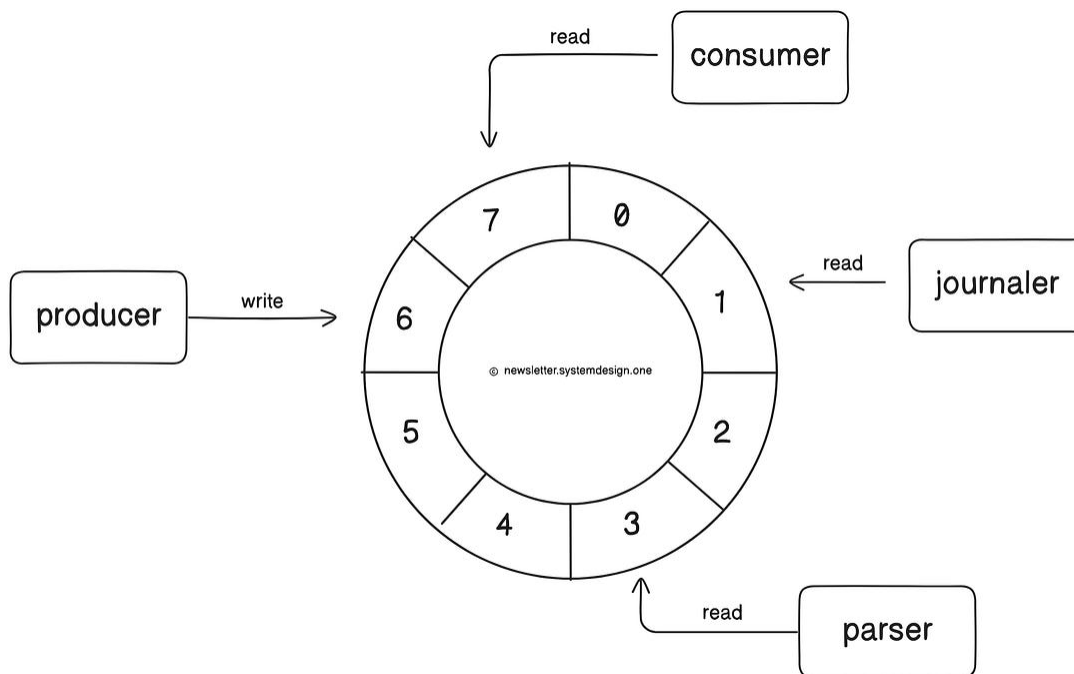
They use compare-and-swap (**CAS**) to claim the sequence number (for the next slot).

Let's dive in...

- A producer takes the next sequence number only if no one else has taken it.
- If another producer wins first, CAS “fails safely” and then retries.
- Plus, CAS runs directly on the CPU hardware and doesn't involve the operating system (no kernel calls, thread sleep/wake). So it's much faster than locks.

But CAS still takes longer than a single thread with no contention.

So a stock exchange keeps ONLY disruptors with a single producer on the critical path. This prevents contention and keeps latency within microseconds.

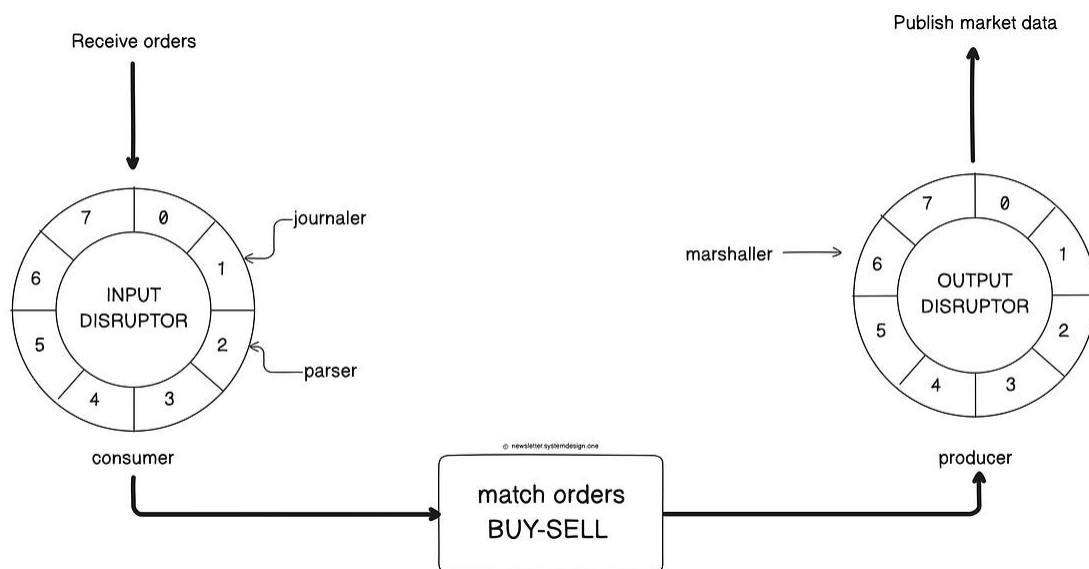


Now, let's look at **how consumers read data from the disruptor**:

- A consumer never reads directly from the ring buffer. Instead, it goes through a ConsumerBarrier provided by the ring buffer.

- Every consumer keeps its own sequence number, so it knows which event to read next.
- Consumers fetch events in batches rather than polling repeatedly. Thus reducing latency spikes.
- Many consumers can read in parallel without blocking each other. No extra queues or coordination needed.

Bringing It Home



- Inbound disruptor (sits between gateway and matching engine):
 - receives orders
 - parse messages
 - journals each order to disk
 - replicates orders to standby servers
 - hands orders to the matching engine in strict order
 - purpose: parallel tasks, no locks, deterministic, high throughput

- Outbound disruptors (sits in matching engine; one per topic: executions, trades, market data):
 - marshal outgoing messages
 - send execution reports
 - send market data updates
 - purpose: handle many consumers, ordered delivery, low latency
 - Sequencing layer:
 - assigns a global sequence number to every incoming event
 - ensures outgoing events stay in the exact same order
 - disruptor gives a single-writer, lock-free sequencing path
 - purpose: fairness, replay, failover
 - Event journaling:
 - events go into a ring buffer before hitting the disk
 - journaling thread drains and writes them to log files
 - purpose: smooth disk IO, no stalls, recovery
-

TL;DR

“Disruptor allows lock-free sequencing and eliminates queue contention.”

- Disruptor is an open-source Java library designed by LMAX to handle a massive number of events with extremely low latency.
- It's used across various components of a stock exchange: Gateway and Matching Engine.
- Since there is no contention, it avoids locks entirely.

- It uses a ring buffer, which is cache-friendly... and avoids garbage collection by reusing event objects (it doesn't allocate new memory during runtime).
- Disruptor prevents false sharing by keeping each thread's data in separate memory chunks, so they don't sit on the same cache line and collide.
- Every producer and consumer has its own sequence counter, so there is no shared head/tail pointer and no blocking.
- Also, events stay in predictable array positions, which keeps them warm in the CPU cache, making access faster.
- Disruptor uses CAS only when many producers need the same sequence number (e.g., non-critical paths like analytics).

Disruptor lets threads pass events with ~0 (sub-microsecond) latency at scale.

Let's dive deep into how it works, with many stocks traded in future newsletters.

Neo Kim



👉 Find me on [LinkedIn](#) | [Twitter](#) | [Instagram](#)

References

- <https://lmax-exchange.github.io/disruptor/files/Disruptor-1.0.pdf>
 - <https://github.com/LMAX-Exchange/disruptor/wiki/Blogs-And-Articles>
 - <https://www.infoq.com/presentations/lmax-trading-architecture/>
 - <https://martinfowler.com/articles/lmax.html>
 - <https://www.baeldung.com/lmax-disruptor-concurrency>
 - <https://mechanitis.blogspot.com/2011/06/dissecting-disruptor-whats-so-special.html>
 - <https://mechanitis.blogspot.com/2011/06/dissecting-disruptor-how-do-i-read-from.html>
 - <https://lmax-exchange.github.io/disruptor/user-guide/index.html>
 - [Concurrent Programming with the Disruptor](#)
 - trishagee.com: [Why it's so fast](#), [Demystifying Memory Barriers](#), [How do I read from the ring buffer?](#), [Wiring up the dependencies](#), [Writing to the ring buffer](#), [What's so special about a ring buffer?](#)
-

- 1 A deadlock occurs when two threads hold locks that the other thread needs. Each thread waits for a lock that will never be released. Since no thread can continue, the system becomes permanently stuck.
- 2 When the CPU needs a value, it loads the entire cache line containing that value into its cache. This means nearby data comes along with it, even if the program only needs one variable.
- 3 False sharing occurs when two threads update different variables that sit on the same CPU cache line. Even though the threads are not actually sharing data, each update forces the other thread's cache line to be invalidated and reloaded. This constant cache-line "ping-pong" slows the system down dramatically.
- 4 Access is predictable as it's an array. Also, the CPU can preload data into the cache. And there's no random memory access like in linked lists.

- 5 The modulo becomes a fast bitmask operation.
- 6 Disruptor creates the ring buffer upfront with a fixed size (for example, 1024 slots). After buffer creation, it immediately fills every slot with an empty event object. So there is no new object creation when events are published.

Why do this? Because creating objects repeatedly causes:

- Memory allocations
- Garbage collection
- CPU stalls
- Unpredictable latency (BAD for stock exchanges)

Pre-allocating everything means:

- Zero allocations during runtime
 - No garbage collection interruptions
 - Ultra-low level latency
- 7 There's a pointer to the next available slot. And each thread writes only to its own sequence number, so there's no contention.
 - 8 Their dependencies are coordinated using a sequence graph.
 - 9 Instead of handling one message at a time, the Disruptor allows a consumer to read many messages in a single go. This means fewer jumps, fewer checks, and better use of the CPU cache. Thus reducing latency and boosting throughput.
 - 10 Mechanical sympathy means writing software that matches how the hardware actually works, so the program runs faster and avoids unnecessary slowdowns.